Security Audit Report

Divigent

v1.0

May 5, 2026

Table of Contents

Table of Contents 2
License 3
Disclaimer 4
Introduction 5
Purpose of This Report 5
Codebase Submitted for the Audit 5
Methodology 6
Functionality Overview 6
How to Read This Report 7
Code Quality Criteria 8
Summary of Findings 9
Detailed Findings 11
1. Insufficient gas budget for the Morpho view call locks the Morpho leg of the protocol 11
2. Incorrect baseline reset enabling artificial yield spike and TWAR manipulation 11
3. Insufficient precision in the Morpho share-price probe produces stair-stepped yield rates 12
4. Inflating share prices leads to DoS of small deposits and loss of funds for depositors 13
5. Operator approvals can be pre-committed before wallet registration 15
6. Deposits lack a caller-supplied minimum-shares slippage guard 15
7. Aave's utilization threshold is not actually checked when depositing 16
8. Fee is rounded against the protocol 17
9. depositWithPermit can be frontrun, leading to an unnecessary revert 17
10. Inconsistent preview function leads to failing withdrawals 18
11. Unsafe Aave calls lead to DoS of deposit or withdrawal, even if Morpho functions as intended 19
12. Approvals on the non-transferable dvUSDC token mutate the allowance mapping despite transfers being blocked 20
13. lastOptimalVaultType is initialized once and never updated thereafter 20
14. Redundant elapsed > 0 check inside the recordObservation function 21
15. Operator approvals require an on-chain transaction from the wallet 21
16. Redundant fee computation when the actual yield is zero 22
17. Oracle will use single or limited observations for the first 4 hours 22
18. Morpho rewards and Aave incentives are not claimable 23
19. APY is calculated without leap years for AAVE, but they are taken into account for Morpho 23
20. MORPHO_VAULT.convertToAssets reverts block views, deposits, and withdrawals 24

License

THIS WORK IS LICENSED UNDER A CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE.

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUE ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

https://oaksecurity.io/
info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by Beyer Ventures SL to perform a security audit of the Divigent smart contracts.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	https://github.com/Divigent/divigent-protocol
Commit	`3656c81d4897b9b1a4ed48a0633473ec97b3d92d`
Scope	All contracts were in scope.
Fixes verified at commit	`5062f7e7fa35110fea6de7ee1210b5fb496c37aa` Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes, such as additional features, have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
    a. Race condition analysis
    b. Under-/overflow issues
    c. Key management vulnerabilities
4. Report preparation

Functionality Overview

Divigent is a non-custodial yield infrastructure for AI agents holding USDC on Base.

Divigent intercepts idle capital intervals in agent payment workflows and deploys them into audited DeFi lending protocols (Aave V3, Morpho Steakhouse USDC), generating yield proportional to idle duration.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, **Partially Resolved**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	Medium	The codebase integrates with third-party contracts.
Code readability and clarity	Medium-High	-
Level of documentation	High	The client provided extensive documentation.
Test coverage	Medium-High	Test coverage reported by `forge coverage` is `98.04%`

Summary of Findings

No	Description	Severity	Status
1	Insufficient gas budget for the Morpho view call locks the Morpho leg of the protocol	Major	Resolved
2	Incorrect baseline reset enabling artificial yield spike and TWAR manipulation	Major	Resolved
3	Insufficient precision in the Morpho share-price probe produces stair-stepped yield rates	Major	Resolved
4	Inflating share prices leads to DoS of small deposits and loss of funds for depositors	Major	Resolved
5	Operator approvals can be pre-committed before wallet registration	Minor	Resolved
6	Deposits lack a caller-supplied minimum-shares slippage guard	Minor	Resolved
7	Aave's utilization threshold is not actually checked when depositing	Minor	Resolved
8	Fee is rounded against the protocol	Minor	Resolved
9	`depositWithPermit` can be frontrun, leading to an unnecessary revert	Minor	Resolved
10	Inconsistent preview function leads to failing withdrawals	Minor	Resolved
11	Unsafe Aave calls lead to DoS of deposit or withdrawal, even if Morpho functions as intended	Minor	Resolved
12	Approvals on the non-transferable `dvUSDC` token mutate the allowance mapping despite transfers being blocked	Informational	Acknowledged
13	`lastOptimalVaultType` is initialized once and never updated thereafter	Informational	Acknowledged
14	Redundant `elapsed > 0` check inside the recordObservation function	Informational	Acknowledged
15	Operator approvals require an on-chain transaction from the wallet	Informational	Acknowledged
16	Redundant fee computation when the actual yield	Informational	Acknowledged

	is zero		
17	Oracle will use single or limited observations for the first 4 hours	**Informational**	**Acknowledged**
18	Morpho rewards and Aave incentives are not claimable	**Informational**	**Acknowledged**
19	APY is calculated without leap years for AAVE, but they are taken into account for Morpho	**Informational**	**Acknowledged**
20	`MORPHO_VAULT.convertToAssets` reverts block views, deposits, and withdrawals	**Informational**	**Acknowledged**

Detailed Findings

1. Insufficient gas budget for the Morpho view call locks the Morpho leg of the protocol

Severity: Major

In `src/DivigentVaultRouter.sol:1015`, the `_planWithdrawCapacity` function calls `MORPHO_VAULT.convertToAssets` with a fixed gas stipend of `MORPHO_VIEW_GAS`, defined as `100_000`, causing the `withdraw` function to revert whenever the supplied gas is insufficient to complete the call.

A cold call to the `convertToAssets` function on the Steakhouse USDC MetaMorpho vault on Base mainnet, measured at block `44,870,973`, consumes approximately `220,000` gas. The configured stipend of `100_000` is therefore well below the cold-path cost, and the call reliably runs out of gas.

The default `forge test` mode does not surface the issue because storage stays warm across `_deposit` and `_withdraw` within a single test function, keeping Morpho's storage slots cached at the time of the view call. Running fork-based withdraw tests with `forge test --isolate`, which executes each external call inside a fresh transaction context, exercises the cold-path cost, and reliably reverts with `MorphoUnreachable`.

Once the router holds Morpho exposure, every production `withdraw` transaction reverts with `MorphoUnreachable`, and funds are trapped on the Morpho leg.

Recommendation

We recommend raising `MORPHO_VIEW_GAS` to a value above the cold-path cost of `convertToAssets`, for example `350_000`.

Additionally, the ability to update the gas stipend through a governance-controlled setter function would allow the protocol to adapt to future changes in Morpho's view cost.

Status: Resolved

2. Incorrect baseline reset enabling artificial yield spike and TWAR manipulation

Severity: Major

In `contracts/src/DivigentYieldOracle.sol:291-324`, the Morpho rate is derived from the difference between `currentSharePrice` and `lastMorphoSharePrice`. When the share price decreases or remains unchanged (`currentSharePrice <= lastMorphoSharePrice`), `newMorphoRate` is set to zero.

However, `lastMorphoSharePrice` is unconditionally updated to the lower `currentSharePrice`. This introduces an asymmetric state update: negative price movements are ignored in rate computation, but the baseline is still reset.

A malicious actor or adverse market fluctuation can exploit this by inducing a temporary price drop followed by a recovery. When the price recovers, the rate calculation uses the artificially lowered baseline, causing the full recovery delta to be annualized within a single interval.

This results in an inflated `morphoSpotRate`, which propagates into downstream calculations such as TWAR.

For example, assume `lastMorphoSharePrice = 1.05e6` (`1.05 USDC`). After a `1%` drawdown and recovery over two 5-minute intervals:

- `t = 300s: currentSharePrice = 1.04e6 newMorphoRate = 0, lastMorphoSharePrice = 1.04e6`
- `t = 600s: currentSharePrice = 1.05e6 Δ = 1e4, elapsed = 300`

```
newMorphoRate = 1e4 × 31_557_600 × 1e27 / 1.04e6 / 300 ≈ 1.01e30
```

This corresponds to roughly `101000%` APY. Even a single 5-minute interval at this level inflates the 4-hour TWAR to `2100%` APY, easily exceeding typical thresholds (e.g., 50 bps) and triggering aggressive capital reallocation.

Consequently, the system can be manipulated into perceiving exaggerated yield conditions, potentially redirecting deposits toward a vault that recently experienced a loss event. This constitutes an economic manipulation vector that can distort protocol behavior without requiring privileged access.

Recommendation

We recommend enforcing symmetric handling of price movements and preventing downward rebasing of `lastMorphoSharePrice` when no positive yield is realized.

Status: Resolved

3. Insufficient precision in the Morpho share-price probe produces stair-stepped yield rates

Severity: Major

In `src/DivigentYieldOracle.sol:288`, the `recordObservation` function reads the current Morpho share price as `MORPHO_VAULT.convertToAssets(SHARE_UNIT)`, where `SHARE_UNIT` is defined as `1e18`.

Because the underlying USDC has 6 decimals, the returned value is itself a 6-decimal USDC quantity. A single observation tick therefore carries a resolution of approximately `1e-6` of the share price.

With a `MIN_OBSERVATION_INTERVAL` of 5 minutes and realistic supply rates between `3%` and `5%` annually, the per-interval price delta `currentSharePrice - lastMorphoSharePrice` frequently rounds down to `0` for several consecutive intervals and then jumps by a single integer unit on the next, producing a stair-stepped Morpho rate. The Morpho TWAR computed by `_computeTWAR` is then systematically biased: low when the integer step has not yet occurred and high when it has just rolled over.

This biased rate feeds the `morphoWins` decision in the `getOptimalVault` function, where Morpho is selected only when its TWAR exceeds Aave's by `MIN_DIFFERENTIAL_RAY`. Routing decisions can swing back and forth between the two vaults purely due to share-price quantization, even when the true Morpho rate is stable.

Recommendation

We recommend probing the Morpho share price with a higher-precision input, for example, by raising `SHARE_UNIT` to `1e24`.

Status: Resolved

4. Inflating share prices leads to DoS of small deposits and loss of funds for depositors

Severity: Major

In `src/DivigentVaultRouter:864-932`, the contract prices new `dvUSDC` shares against the router's asset value, computed as `A_TOKEN.balanceOf(router) + MORPHO_VAULT.convertToAssets(MORPHO_VAULT.balanceOf(router))`.

Both `aTokens` (`aBasUSDC` on Base) and Morpho vault shares are ERC-20s that can be transferred by third parties, so an attacker can increase `totalVaultAssets` without increasing `dvUSDC` supply.

With the router starting from `totalSupply = 0` (and an effective supply of `1` inside the mint formula because of the `VIRTUAL_OFFSET`), an inflated total can easily affect share minting. If the attacker donates `N` USDC worth of morpho shares or `aTokens`, any later deposit of `A <= N` mints zero `dvUSDC` shares after flooring and reverts, which raises the effective minimum deposit to the attacker's chosen threshold until the pool grows enough to dilute it.

Example:

1. Attacker donates `1001 USD` of `aTokens` to the router while the vault is empty

2. Alice tries to deposit `1000 USDC`
3. Alice would receive `0.99` shares, which is floored to `0` shares; the deposit reverts

The DoS is not the only attack vector. An attacker that sees an initial deposit of size `X` can inflate the price per share to approximately `0.5 * X + 1` using a donation of roughly `0.5 * X`, causing the victim to receive only one share after flooring instead of the roughly two shares they would otherwise obtain. The floored-away value is redistributed across all shareholders, so an attacker that also seeds a large stake of their own recovers almost the entire donation.

Example.

1. Alice initiates a deposit of `1000 USDC` into an empty Divigent vault.
2. Bob front-runs by donating `500 USDC` in `aTokens` to the router. The share price is now `(500 + 1) / (0 + 1) = 501 USDC`.
3. Bob deposits `50,100 USDC` and obtains exactly `50100 * 1 / (500 + 1) = 100` shares. After: `totalAssets = 50,600`, `totalSupply = 100`.
4. Alice's deposit is processed next: `1000 * (100 + 1) / (50,600 + 1) ≈ 1.996` shares, floored to `1`. After: `totalAssets = 51,600, totalSupply = 101.`
5. Each share is now worth `(51,600 + 1) / (101 + 1) ≈ 505.89 USDC`.

As a result, Alice deposited `1000 USDC` and lost `≈ 494 USDC` (`≈ 49%`), while Bob's `100` shares are worth `≈ 50,588.24 USDC`. Against his combined outlay of `500 + 50,100 = 50,600 USDC`, his net cost is `≈ 11.76 USDC`.

In addition, roughly `505.89 USDC` of value that no real holder can claim are absorbed by the virtual offset.

Bob has recovered almost all of his donation, making this a very cheap griefing opportunity. Every additional depositor whose mint floors contribute further value that is captured proportionally by Bob's `100`-share stake (`100/101` of each floor loss), so one additional victim of similar size flips Bob from roughly break-even to clearly profitable. Deposits below `≈ 501 USDC` revert with a `ZeroAmount` error.

Recommendation

We recommend:

- Increasing the virtual offset: Increasing `VIRTUAL_OFFSET` in line `108` reduces per-victim rounding loss because share resolution is finer at larger offsets, which removes the economic viability of the small-deposit DoS and severely reduces the likelihood of the share-flooring griefing.

- Adding slippage control: Add a `minSharesOut` parameter to the deposit function and enforce it via `if (dvUsdcMinted < minSharesOut) revert SlippageTooHigh`, which lets users reject adverse rounding directly.
- Initially, deposit a small amount into the vault atomically right after deployment to hinder any attempt at share price inflation.

Status: Resolved

5. Operator approvals can be pre-committed before wallet registration

Severity: Minor

The `setOperator` function in `src/DivigentVaultRouter.sol:293-297` writes `isOperator[msg.sender][operator] = approved` without checking whether `msg.sender` is a registered wallet. Registration typically occurs via the `initialize` function, which sets `authorizedWallets[msg.sender]` to `true`.

An address that has never called `initialize` may still call `setOperator` and populate the operator mapping, although the wallet was never marked as authorized, resulting in a disjoint state.

Recommendation

We recommend reverting in the `setOperator` function if the caller is not a registered wallet.

Status: Resolved

6. Deposits lack a caller-supplied minimum-shares slippage guard

Severity: Minor

The `_deposit` function in `src/DivigentVaultRouter.sol:898-966` computes the share mint amount as `dvUsdcMinted = _assetsToShares(amount, totalAssetsBefore, totalSupplyBefore)` in line `956` and only validates that the result is non-zero. The function does not accept a caller-supplied minimum, and neither the `deposit` nor the `depositWithPermit` function exposes one.

Between when a wallet quotes a deposit via `previewDeposit` and when `_deposit` executes, `totalVaultAssets` can move adversely. A Morpho bad-debt event, a market closure that drops `convertToAssets`, or a same-block donation followed by reordering can each shift `totalAssetsBefore` and reduce the share count returned by `_assetsToShares`. The deposit completes, but the wallet receives fewer `dvUSDC` shares than the preview implied, with no out-of-band signal to abort.

Recommendation

We recommend extending the `deposit` and `depositWithPermit` functions to include a `minSharesOut` parameter, and reverting in `_deposit` when `dvUsdcMinted < minSharesOut`. This gives callers an explicit slippage bound, mirroring the `minUsdcOut` guard already present in the `withdraw` function.

Status: Resolved

7. Aave's utilization threshold is not actually checked when depositing

Severity: Minor

In `src/DivigentYieldOracle:79`, the contract sets `UTILISATION_THRESHOLD_BPS = 9_000` which limits the maximum utilization of the aave pool, which the vault router will use to `90%`, and in lines `474-477`, `_isVaultSafe(VaultType.AAVE)` returns `false` when Aave's utilization is `90%` or above, which matches the NatSpec note: *"safe if utilisation < 90%."*. This is also implied/mentioned in multiple other places in the codebase/comments. The check is only used on Aave in the oracle's external view function, `getAllRates`.

However, this check is never run on a deposit. `getOptimalVault` only calls `_isVaultSafe(VaultType.MORPHO)` in line `199`, and then falls back to Aave whenever Morpho doesn't provide a `50` bps higher yield. Whether `_isVaultSafe(VaultType.AAVE)` would have returned false is never checked.

Later in the deposit flow, `_canAllocate(VaultType.AAVE, amount)` in `src/DivigentVaultRouter:898-905` is called, which then only checks the `USDC.balanceOf(address(A_TOKEN))` against the deposit amount, which is an idle-liquidity check, not a utilization check. Aave can be near full utilization and still pass `_canAllocate` as long as the idle USDC sitting on the `aToken` exceeds this single deposit.

If utilization is already high at the time of the deposit, idle USDC on the `aToken` is small relative to total supply, and withdrawals routed against the Aave leg can get stuck. The whole point of the threshold is to refuse routing into a pool that's already saturated; checking it at deposit time is what avoids stuck-funds

Recommendation

We recommend calling `_isVaultSafe(VaultType.AAVE)` inside `getOptimalVault` before falling back to Aave.

If it returns `false` and Morpho is safe, route to Morpho. If neither is safe, the router should revert instead.

Status: Resolved

8. Fee is rounded against the protocol

Severity: Minor

When the protocol fee is calculated in `src/DivigentFeeCollector:135`, the fee amount is floored, leading to a small `1e-6 USDC` loss for the protocol on every withdrawal as the user underpays.

Plain integer division truncates toward zero, so any `yieldEarned * FEE_BPS` not exactly divisible by `BPS_DENOMINATOR` sheds the remainder in the user's favor.

Recommendation

We recommend switching to ceiling rounding instead, to follow best practice recommendations and provide defense in depth

Status: Resolved

9. `depositWithPermit` can be frontrun, leading to an unnecessary revert

Severity: Minor

An attacker can front-run the `USDC.permit` call in `src/DivigentVaultRouter:336`. While this still only allows the `DivigentVaultRouter` to spend users' funds, not the attacker's, it causes `depositWithPermit` to revert, blocking this deposit path.

Since `USDC.permit` uses a nonce to protect against replays of the same signature, and the signature is visible in the public mempool, any attacker can call `USDC.permit` before the user calls `depositWithPermit` and causes the `USDC.permit` call within `depositWithPermit` to fail as the nonce has already been consumed. The user can create another signature, but it can be front-run again indefinitely. This does not affect the regular deposit path, which the user can still use.

The raw `permit` call is not wrapped in a `try/catch` block, so a front-run that consumes the nonce in USDC causes a full revert of `depositWithPermit`, rather than being swallowed when the allowance is already in place.

Recommendation

We recommend wrapping the permit signature in a try-catch block as recommended by OpenZeppelin.

Status: Resolved

10. Inconsistent preview function leads to failing withdrawals

Severity: Minor

Two read-only helpers in `src/DivigentVaultRouter:530` and `618` return values that don't match what the state-changing functions actually use. An agent that picks its withdrawal size from these getters (`pricePerShare` and `previewWithdrawNet`) ends up with a USDC amount the router refuses to deliver, or a preview that the next `withdraw` reverts on. Divigent is meant to back AI agents settling payments over x402 on Base, so the main caller of these views is autonomous software running a deposit-withdraw loop.

This is caused by the views using different formulas from the execution path. Mints and redemptions go through `_assetsToShares` and `_sharesToAssets`, which add `VIRTUAL_OFFSET` to both sides of the ratio, and `withdraw` enforces a ceiling at `walletShares`. The views do neither. `pricePerShare` returns the plain ratio in line `530`, while the execution path uses `(assets + VIRTUAL_OFFSET) * 1e18 / (supply + VIRTUAL_OFFSET)`, so the numbers never match.

The `previewWithdrawNet` function is documented as returning the minimum shares needed to receive at least a target net USDC, but it silently clamps to `walletShares`, leading to an amount of shares being returned that cannot actually deliver the expected USDC when the wallet owns too few shares:

`withdraw` then runs its own slippage check and reverts with `SlippageExceeded` when the delivered amount falls short of `minUsdcOut`, so `previewWithdrawNet(x)` followed by `withdraw(shares, wallet, x)` reverts even when the caller used the router's own preview. `previewWithdrawNet` also returns 0 for four different states (request was zero, wallet has no shares, wallet has shares that round to zero USDC, profit-branch denominator collapses to zero) with no way to tell them apart.

Recommendation

We recommend changing the `pricePerShare` function to `(totalVaultAssets() + VIRTUAL_OFFSET) * 1e18 / (totalSupply() + VIRTUAL_OFFSET)` and making the zero-supply branch match `_sharesToAssets` at that state.

In the `previewWithdrawNet` function, drop the clamp and revert with a typed error like `UnservicableNet(desiredNetUSDC, maxDeliverable)` when the request exceeds what the wallet can deliver, and split the four 0 returns into distinct reverts.

Status: Resolved

11. Unsafe Aave calls lead to DoS of deposit or withdrawal, even if Morpho functions as intended

Severity: Minor

In `src/DivigentVaultRoute:831-832` and `970-974`, the availability of Aave for deposits and withdrawals is checked by assessing idle USDC in the `aToken` contract and assets already held on the Aave leg, but it does not read Aave's reserve configuration flags before planning deposits or withdrawals.

That means the router treats Aave as available even when the reserve is paused, frozen, or inactive. The planning logic then routes part of the operation through Aave because the numeric balance checks still look healthy, only to hit the real Aave call later and revert.

The same blind spot affects `withdrawCapacity`, which can report non-zero capacity even when Aave itself is unavailable.

This breaks both sides of the user flow. On deposits, if the oracle prefers Aave, the router can fail to fall back to Morpho even though Morpho is healthy, because the pre-check never recognizes that Aave has stopped accepting supply.

On withdrawals, as long as the vault has any Aave exposure, users will be unable to exit during an Aave pause because the planner allocates some portion to the Aave leg and never redirects it away before the revert.

Deposit-side, `_canAllocate` in lines `831-832` only checks idle USDC on the `aToken` and never reads the reserve flags:

Withdraw-side, `_planWithdrawCapacity` derives `aaveWithdrawCap` in lines `970` to `974` from balances only, so it stays positive while Aave is paused or frozen, and the unguarded `AAVE_POOL.withdraw` then reverts.

Recommendation

We recommend decoding the reserve flags from `AAVE_POOL.getReserveData(USDC).configuration` (bit `56` active, bit `57` frozen, bit `60` paused) and gate both legs on them. Supply requires active and not frozen and not paused; withdraw requires active and not paused.

In `_canAllocate` returns `false` on the Aave branch when supply is disallowed, and in `_planWithdrawCapacity` sets `aaveWithdrawCap = 0` when withdraw is disallowed. The project's `IAaveV3Pool` returns the fields flat, so the actual call destructures the first return value: `(uint256 configuration, , , , , , , , , , , , , , , ) = AAVE_POOL.getReserveData(address(USDC));`

Status: Resolved

12. Approvals on the non-transferable `dvUSDC` token mutate the allowance mapping despite transfers being blocked

Severity: Informational

The `_update` function override in `src/dvUSDC.sol:78-87` reverts on any non-mint, non-burn movement and enforces the contract's non-transferability.

However, the override does not extend to the inherited `approve` and `_approve` flow. A wallet may still invoke `approve` with a non-zero value and write an entry into the `_allowances` mapping. The subsequent `transferFrom` reverts inside `_update` with `NonTransferable`, so no token actually moves.

The leftover allowance, while harmless to balances, remains visible to block explorers, wallet interfaces, and approval-scanner tools. End users may then see stale approvals associated with a non-transferable token and attempt to revoke them, only to discover that the action offers no real protection.

Recommendation

We recommend overriding `_approve` in the `DvUSDC` contract to revert to `NonTransferable` whenever a non-zero allowance is set.

Status: Acknowledged

13. `lastOptimalVaultType` is initialized once and never updated thereafter

Severity: Informational

The `lastOptimalVaultType` state variable in `src/DivigentYieldOracle.sol:139` is declared as a public storage slot. It is assigned once, in the `constructor`, where it is set to `VaultType.AAVE` as a conservative default.

The `recordObservation` function in lines `268-329`, where the value would naturally be refreshed alongside the rate update, never writes to it. The variable, therefore, does not reflect the most recently selected optimal vault and behaves as a constant.

The variable is publicly accessible. External integrators that might read it, expecting a current routing signal, observe a stale value indefinitely.

Recommendation

We recommend either updating `lastOptimalVaultType` within `recordObservation` or removing the variable entirely if no consumer requires it.

Status: Acknowledged

14. Redundant `elapsed > 0` check inside the `recordObservation` function

Severity: Informational

The `recordObservation` function in `src/DivigentYieldOracle.sol:268-329` rate-limits observation updates with the early return `if (elapsed < MIN_OBSERVATION_INTERVAL) return;` in line `272`. By construction, after this guard `elapsed >= MIN_OBSERVATION_INTERVAL`, and `MIN_OBSERVATION_INTERVAL` is defined as `5 minutes` in line `65`, so elapsed is always strictly positive past this point.

The `elapsed > 0` check in line `294`, used as part of the new Morpho rate computation, is therefore unreachable in its negative form.

Recommendation

We recommend removing the `elapsed > 0` check. The remaining checks already capture the meaningful preconditions for computing the new Morpho rate.

Status: Acknowledged

15. Operator approvals require an on-chain transaction from the wallet

Severity: Informational

The `setOperator` function in `src/DivigentVaultRouter.sol:293-297` is the only path to authorize an operator on a wallet's behalf, and it relies on `msg.sender` to identify the wallet. The wallet must therefore submit the transaction and pay gas fees.

The protocol already supports gasless onboarding through the `initializeFor` function in lines `267-290`, which accepts an EIP-712 signature and allows a relayer to submit the registration on the wallet's behalf. The same pattern is not available for operator approvals.

Recommendation

We recommend introducing a `setOperatorWithPermit` function that accepts an EIP-712 signed authorization from the wallet and updates `isOperator[wallet][operator]` accordingly, mirroring the structure of `initializeFor`.

Status: Acknowledged

16. Redundant fee computation when the actual yield is zero

Severity: Informational

In `src/DivigentVaultRouter.sol:495-508`, the `withdraw` function computes `actualYield`, `feeAmount`, and `usdcReturned` and then enters a guarded fee-collection branch. When `actualGross <= principalOut`, line `495` clamps `actualYield` to `0`, the call to `FEE_COLLECTOR.calculateFee(actualYield)` in line `496` returns `0`, and the outer `if (feeAmount > 0)` in line `504` skips the call to `FEE_COLLECTOR.collectFee`. The `DivigentFeeCollector` contract also short-circuits internally when `yield` is zero.

The intermediate computation in line `496`, therefore, produces a guaranteed-zero result that is checked twice before any meaningful action is taken, resulting in unnecessary gas expenditure.

Recommendation

We recommend computing `feeAmount` only when `actualYield > 0`, for example, by guarding the `calculateFee` call with the same condition that already guards `collectFee`.

Status: Acknowledged

17. Oracle will use single or limited observations for the first 4 hours

Severity: Informational

In `src/DivigentYieldOracle:367-437`, the oracle presents itself as ready before it has accumulated enough history to behave like the TWAR-based component its interface implies. Freshness is determined by how recently an observation was recorded, not by whether the internal checkpoint buffer spans the full configured averaging window. As soon as the first observation is written, `isFresh` can return true, while `_computeTWAR` still has no mature window to calculate a time-weighted average.

This leads to a warm-up period, while `timeSinceDeployment < TWARWindow` (or after history has otherwise been reset), during which users and integrators may believe the router is using time-weighted rates when it is actually using a shorter time interval.

Recommendation

We recommend either adding documentation about the fact that the Oracle will use limited observations during the warm-up period(s) or blocking deposits for the TWAR window duration (4 hours) after deployment.

Status: Acknowledged

18. Morpho rewards and Aave incentives are not claimable

Severity: Informational

In `src/DivigentVaultRouter:914-917`, the contract supplies to Aave with `address(this)` as the `aToken` recipient and deposits into the MetaMorpho vault with `address(this)` as the share owner, so the router is the on-chain holder of any incentive rewards that Aave's rewards controller or Morpho's Universal Reward Distributor attributes to the router's positions.

The router contains no `claim`, `skim`, or `sweep` function, or integration with either Aave's `RewardsController` or Morpho's URD, and every contract in the stack is immutable/non-upgradeable. Anything a third-party incentive program credits to the router address during the contract's lifetime is therefore unreachable.

At the time of this review, there are no active Morpho rewards programs or Aave incentive programs affecting this protocol, but new ones may launch, or existing programs may be reinstated at any time.

Recommendation

We recommend either adding functionality to claim these incentives and rewards if reward or incentive programs are launched or reinstated, or clearly documenting that these potential additional yield sources will not be claimable.

Status: Acknowledged

19. APY is calculated without leap years for AAVE, but they are taken into account for Morpho

Severity: Informational

In `src/DivigentYieldOracle:335-355`, the `currentLiquidityRate` is queried from Aave V3, which is calculated by multiplying by `SECONDS_PER_YEAR = 365`, (see Aave MathUtils), while the calculation the protocol uses for Morpho uses `SECONDS_PER_YEAR = 365.25` to account for leap years.

This biases the decision in which protocol to invest in Morphos direction by `APY * 0.000685`. The impact of this is limited to small yearly yields but increases with APY and can lead to sub-optimal deposit decisions.

Recommendation

We recommend using `SECONDS_PER_YEAR = 365` to prevent any biases towards one of the yield-bearing protocols.

Status: Acknowledged

20. `MORPHO_VAULT.convertToAssets` reverts block views, deposits, and withdrawals

Severity: Informational

`MORPHO_VAULT.convertToAssets` is called without a try/catch block in both the `morphoAssetsHeld` function in `src/DivigentVaultRouter.sol:938-942`, and in `src/DivigentYieldOracle.sol:463-485`.

`_morphoAssetsHeld` is used in `totalVaultAssets`, which in turn is used for the deposit side TVL check.

`_isVaultSafe` in `DivigentYieldOracle:482`, which is called by `getOptimalVault` in the deposit path.

If `convertToAssets` reverts (paused underlying market, curator reallocation, bad-debt edge case), `totalVaultAssets` cannot return, and `getOptimalVault` cannot return. Frontends cannot quote `pricePerShare`, run any preview helper, or read `getPosition`. Deposits revert because the TVL-cap check pulls `totalVaultAssets`, and routing cannot fail over to Aave because the safety check itself reverts.

For withdrawals `_planWithdrawCapacity` in line `965` already wraps the same Morpho view in a gas-bounded try/catch, but the withdraw function then reverts in line `399`: `if (!cap.morphoReachable) revert MorphoUnreachable();`

Aave and Morpho share a single pool, and all withdrawals are paid out proportionally, so a Morpho view failure blocks all withdrawals regardless of the available Aave liquidity.

We classify this issue as informational because a naive try/catch on the two strict reads would allow users to redeem `dvUSDC` against an undercount of the vault's true assets, resulting in partial withdrawals at a price that does not reflect the Morpho leg and breaking share accounting.

Recommendation

We recommend considering adding an explicit emergency-exit mode that lets a user opt into a haircut withdrawal of only the Aave leg when Morpho is unreachable, rather than broadening the strict reads.

Status: Acknowledged